



Cloud Specialist
Unità Formativa (UF): Containers Microservizi
Serverless
Docente: Denis Maggiorotto
Titolo argomento: Introduzione a Docker

Il docente

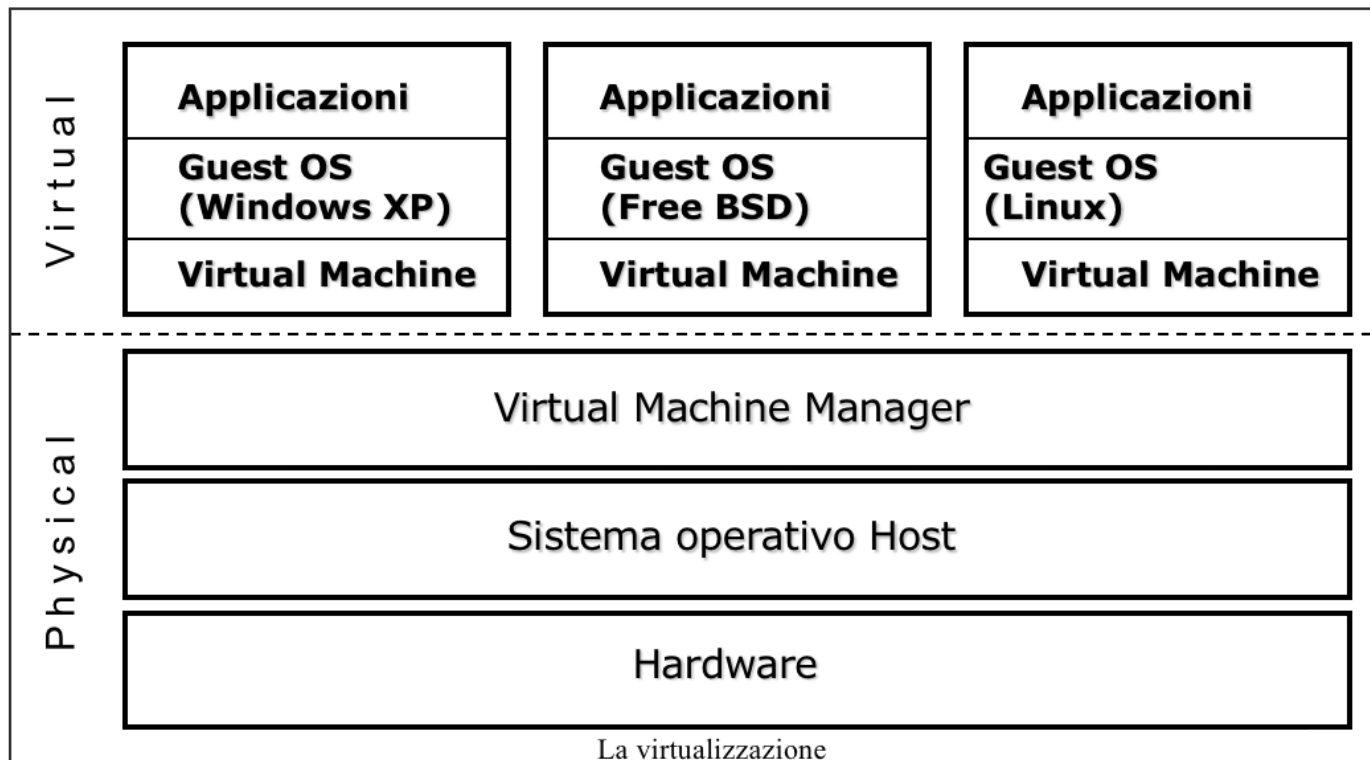


Denis Maggiorotto

Denis Maggiorotto

- Managing Director and shareholder @ **Sunnyvale S.r.l.**
- 20 years of experience in ICT consulting
- Senior Software / Enterprise Architect @ Major companies in public utility, telco, TV broadcasting and banking sector
- Oracle University Principal Instructor regarding Java technologies (Micro Edition, Standard Edition and Enterprise Edition) and Oracle's middleware products.
- Independent IT professional trainer and public speaker

Sistemi virtuali



Denis Maggiorotto



denis.maggiorotto@its-ictpiemonte.it



[@denismaggior8](https://twitter.com/denismaggior8)



<https://www.linkedin.com/in/denismaggiorotto/>



<https://github.com/denismaggior8>

Introduzione a Docker



...partendo dalla virtualizzazione del SO

Tipologie di virtualizzazione

Emulazione

Permette l'esecuzione di un SO su una CPU completamente differente

Virtualizzazione piena (VirtualBox, Qemu)

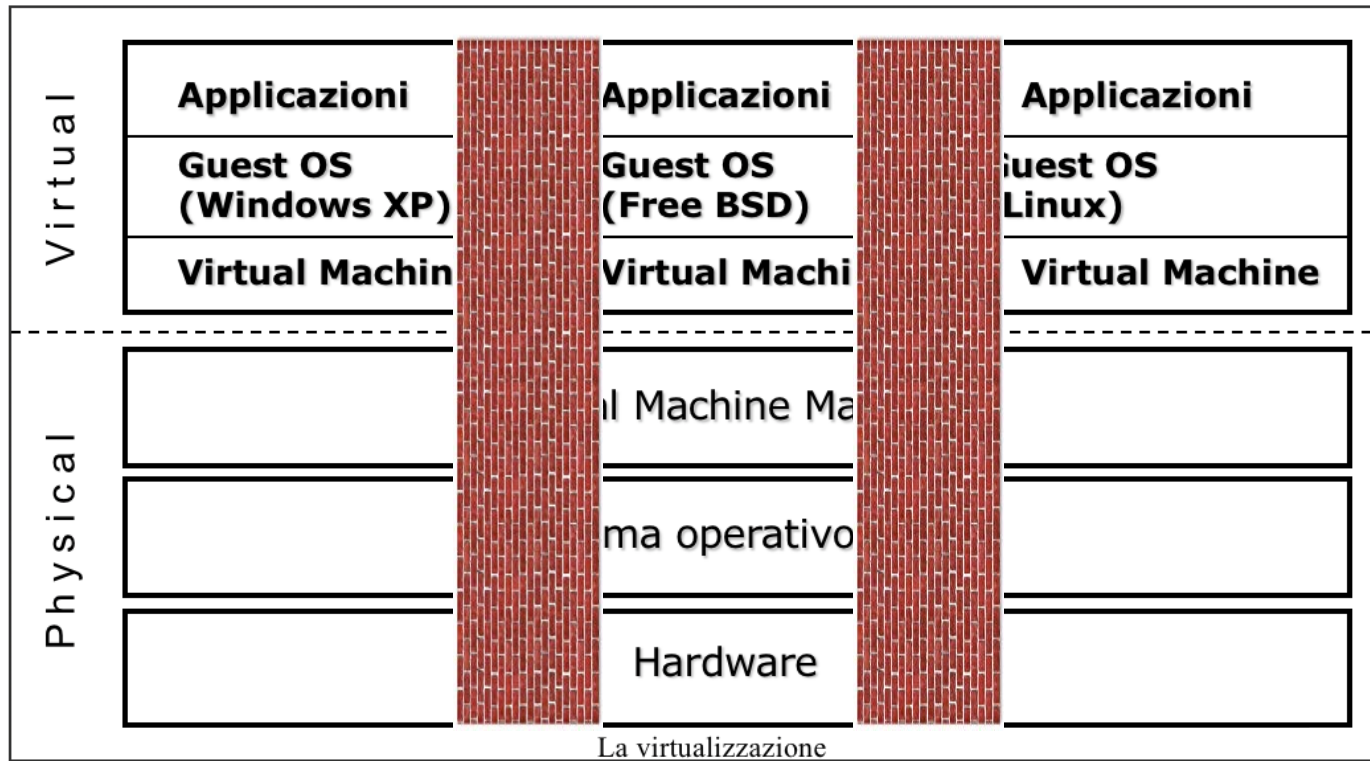
Esegue copie di SO completi

Poco efficiente

Para-virtualizzazione (UML, Xen)

Esegue SO completi su architetture particolari (es: XEN/x86)

Segregazione delle risorse



Vantaggi della virtualizzazione

Multi SO: Più SO sulla stessa macchina fisica, più ambienti di esecuzione (eterogenei).

Sicurezza: eventuali attacchi da parte di malware o spyware sono confinati alla singola macchina virtuale

In ambito didattico: invece di assegnare ad ogni studente un account su una macchina fisica, si assegna una macchina virtuale.

Risorse: Segregazione delle risorse della macchina fisica (Host) su ciascuna macchina virtuale

Difetti della virtualizzazione

Spreco di risorse: le macchine virtuali possono occupare un bel po' di risorse hardware: ogni macchina virtuale non esegue solo una copia di un certo sistema operativo ma sfrutta una copia virtuale di tutto l'hardware di cui lo stesso sistema operativo necessita per funzionare correttamente.

Un approccio che necessita di RAM e che occupa parecchi cicli di CPU. Sempre più economico se paragonato all'utilizzo di più macchine a sé stanti ma gravoso dal punto di vista dell'occupazione delle risorse in alcuni frangenti.

Portabilità: Le tecnologie di virtualizzazione oggi presenti sul mercato non sono del tutto interoperabili, rendendo così le VM poco portabili tra prodotti di virtualizzazione differenti

I containers

Perché virtualizzare un'intera macchina, quando sarebbe possibile virtualizzare solamente una piccola parte di essa?

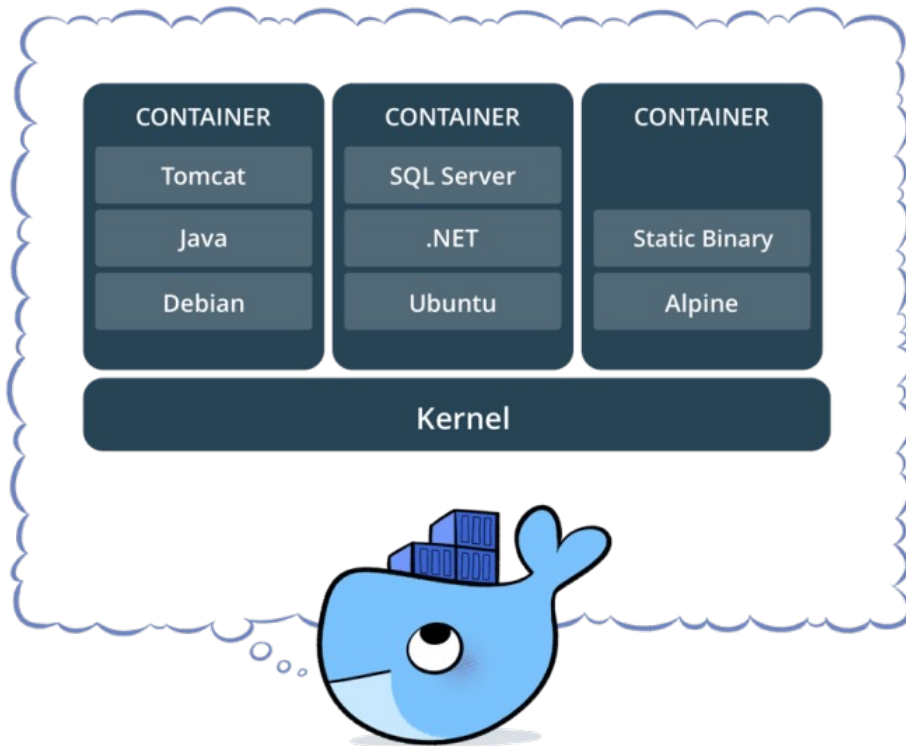
I container sono una sorta di sistema operativo semplificato e virtualizzato su quello ospitante, insieme a tutti i dati di cui necessita un'applicazione per essere eseguita: librerie, altri eseguibili, rami del file system, file di configurazione e script.

Ogni container vive in un contesto di esecuzione isolato, con un alto livello di astrazione e segregazione rispetto agli altri container e al sistema operativo sottostante.

Storia dei container qui: <https://www.internetpost.it/container-linux-storia-tecnologia/>



I containers Docker

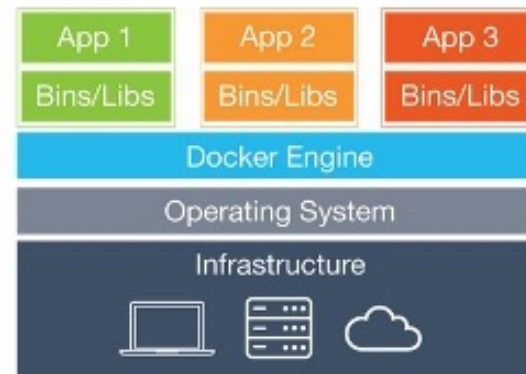


Docker è un progetto open-source che semplifica l'esecuzione di applicazioni all'interno di container, utilizzando le funzionalità di isolamento delle risorse offerte dal kernel Linux, come ad esempio cgroup e namespace, per consentire a container indipendenti di coesistere sulla stessa istanza di Linux, evitando però l'installazione e la manutenzione di una macchina virtuale.

Virtual machines vs containers

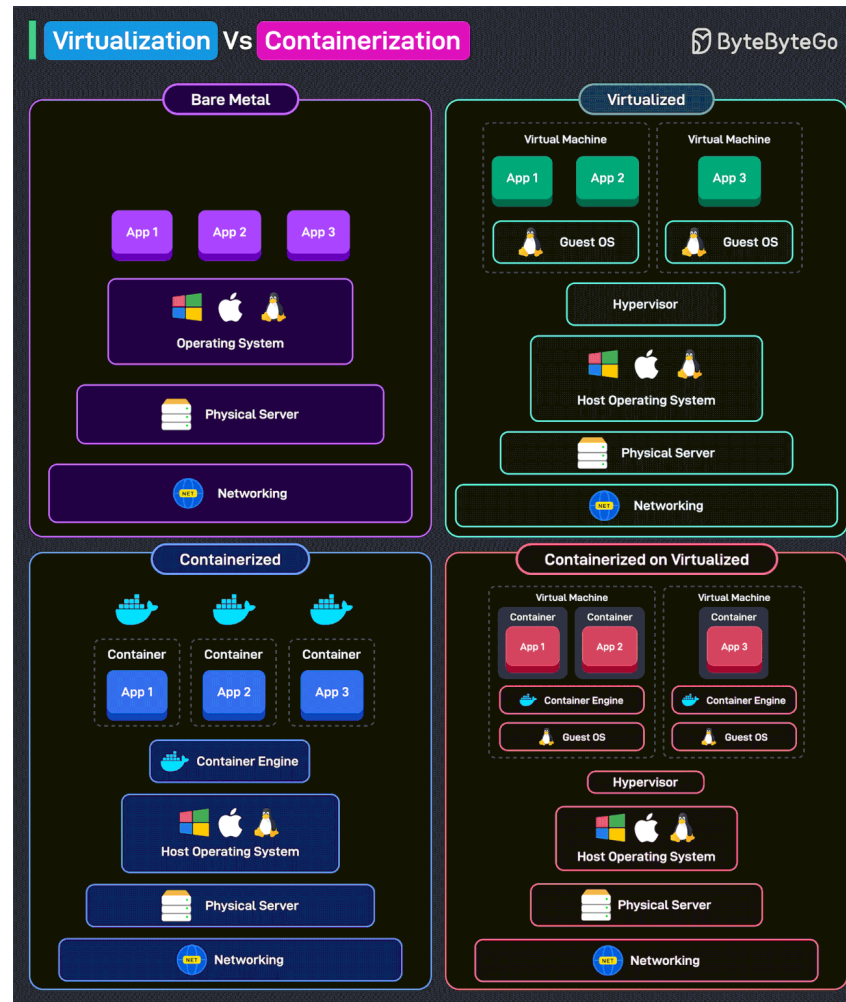


Virtual Machines

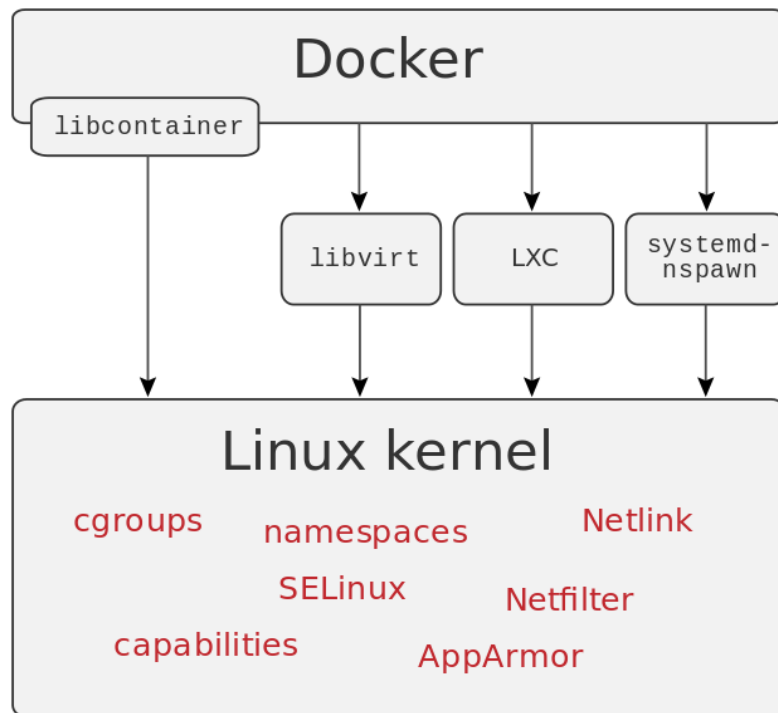


Containers

Virtual machines vs containers



Docker e Linux



libcontainer: fornisce un'implementazione per la creazione di container con namespaces, cgroups, controllo di accesso al filesystem. Consente di gestire il ciclo di vita dei container.

libvirt: libvirt è un'API open source, un demone e uno strumento di gestione di piattaforme di virtualizzazione (KVM, Xen, VMware ESXi, QEMU).

LXC: è un ambiente di virtualizzazione a container, che opera a livello del sistema operativo e permette di eseguire diversi ambienti Linux virtuali isolati tra loro (container).

systemd-nspawn: viene utilizzato per eseguire comandi o un SO in un namespace. Virtualizza completamente la gerarchia del file system, nonché l'albero dei processi, i vari sottosistemi IPC e il nome host e dominio.

cgroups: sono una caratteristica del kernel che consente di controllare e limitare l'utilizzo delle risorse (CPU, memoria, network I/O, disk I/O) da parte di un processo o gruppo di processi.

namespaces: il compito di namespace è quello di permettere l'astrazione delle risorse (IPC, rete, punto di mount di /, l'albero dei processi, gli utenti e i gruppi, risoluzione dei nomi di rete) e isolarle dal resto del sistema.

Vantaggi dei container Docker

Gli stessi di una macchina virtuale, con l'aggiunta di:

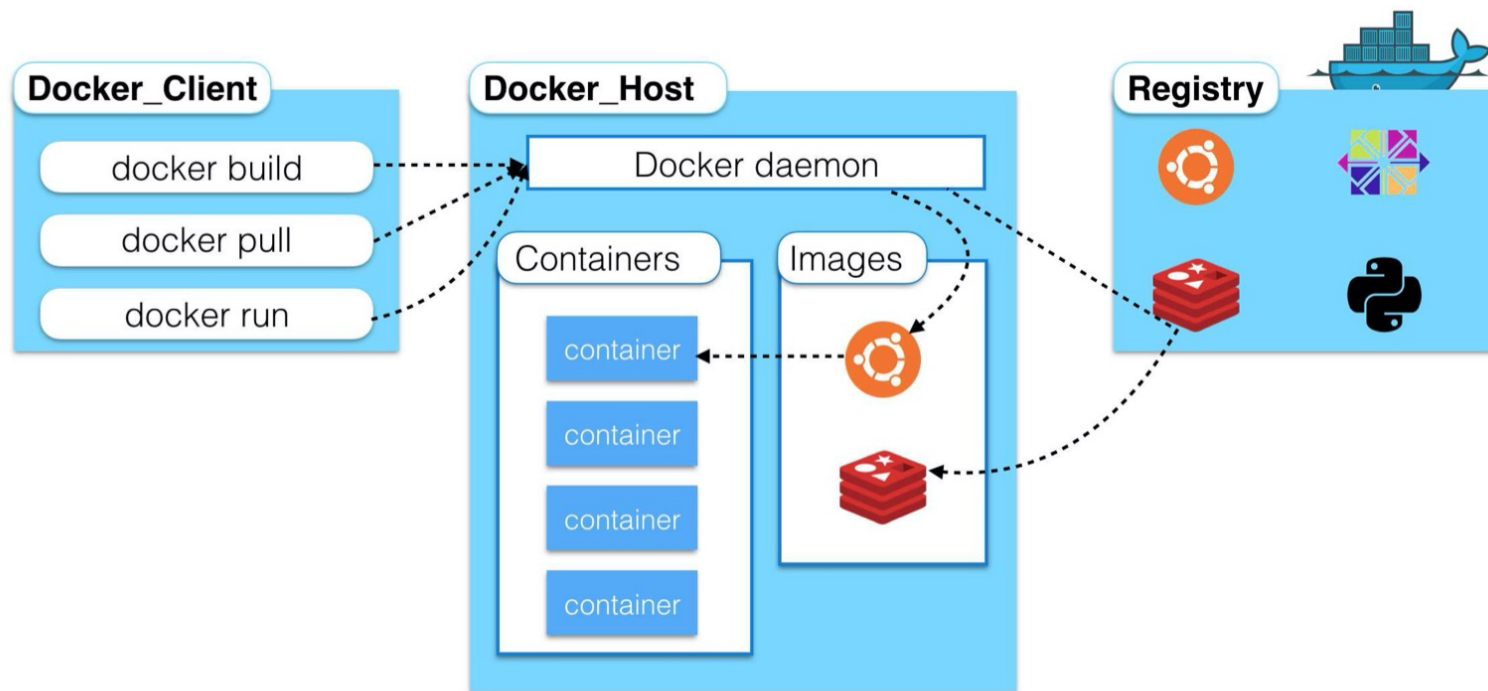
- Portabilità delle immagini
- Dimensioni più contenute
- Risparmio risorse CPU e RAM non dovendo virtualizzare l'intero sistema operativo Guest
- Facilità di distribuzione delle immagini
- Miglior efficienza nella gestione dello spazio disco
- Tempi più ridotti nello startup e shutdown
- Facilità di utilizzo anche in pipeline di CI/CD

Struttura e definizione delle componenti necessarie a creare un Docker container



Containers, immagini, DockerHUB

Architettura di Docker



Componenti di Docker

Docker host: è il server su cui viene installato Docker e dove è presente il runtime necessario ad avviare dei container.

Docker daemon: è il processo che rimane in attesa degli «ordini» da parte del client (docker CLI tool), per poi effettuare le operazioni necessarie ad eseguire o stoppare un container sul Docker host.

docker CLI tool: Si tratta di un tool a linea di comando utilizzato per impartire istruzioni al Docker daemon ed ottenere ad esempio l'esecuzione di container sul Docker host.

Images: rappresentano la matrice dalla quale viene istanziato un container. Le immagini contengono tutto il necessario utilizzato dal container per poter essere avviati ed erogare servizio. Le immagini possono estendere altre immagini e sono di natura immutabili, ovvero per modificare un'immagine è necessario creare un'altra immagine che la estenda.

Container: si tratta di un'istanza creata a partire da un'immagine. Il container è mantenuto in vita da un singolo processo, ovvero l'applicazione che è stata containerizzata.

Docker registry: è un registro centralizzato, esposto su web, dove è possibile scaricare immagini da utilizzare per creare dei container.

Il nostro primo Docker container

```
$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
1b930d010525: Pull complete
Digest: sha256:9572f7cdcee8591948c2963463447a53466950b3fc15a247fcad1917ca215a2f
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
...
```

docker: comando

run: sottocomando

hello-world: immagine. La versione dell'immagine non è stata specificata, default = latest

Sequenza di eventi

- 1) L'utente digita il comando **docker run**
- 2) Il client (docker) invoca il run del container al Docker daemon
- 3) Il Docker daemon, ricevuto il comando di run, verifica l'assenza dell'immagine hello-world in locale sul Docker host
- 4) Il Docker daemon contatta il Docker registry e scarica l'immagine hello-world sul Docker host
- 5) Terminato il download dell'immagine, il Docker daemon istanzia l'immagine ricavandone un container.
- 6) Il container appena avviato stampa a video (standard output) il messaggio «Hello from Docker!»
- 7) Il container, avendo esaurito il suo compito, termina

Alcuni comandi utili

Stampa l'elenco dei container attivi (output vuoto significa nessun container attivo):

```
$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
--------------	-------	---------	---------	--------	-------	-------

Stampa l'elenco di tutti i container creati:

```
$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
0ec32c45c710	hello-world	"/hello"	23 minutes ago	Exited (0)	23 minutes ago	inspiring_borg

Stampa l'elenco delle immagini scaricate

```
$ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
hello-world	latest	fce289e99eb9	13 months ago	1.84kB

Alcuni comandi utili

Start di un container in stato Exited

```
$ docker start 0ec32c45c710  
ec85ba00ff30
```

Stop di un container in stato Up

```
$ docker stop 0ec32c45c710  
ec85ba00ff30
```

Rimozione di un container

```
$ docker rm 0ec32c45c710  
0ec32c45c710
```

Rimozione di un'immagine (tutti i container che la referenziano devono essere rimossi prima)

```
$ docker rmi fce289e99eb9  
Untagged: hello-world:latest  
Untagged: hello-world@sha256:9572f7cdcee8591948c2963463447a53466950b3fc15a247fcad1917ca215a2f  
Deleted: sha256:fce289e99eb9bca977dae136fbe2a82b6b7d4c372474c9235adc1741675f587e  
Deleted: sha256:af0b15c8625bb1938f1d7b17081031f649fd14e6b233688eea3c5483994a66a3
```

Creazione di un'immagine

1

```
FROM ubuntu:15.04
CMD echo "Hello by <NOME> <COGNOME>!"
```



- dichiarazione dell'immagine di base
- istruzioni che verranno eseguite in fase di build al fine di creare la nuova immagine

Ricordati di sostituire
<NOME> e <COGNOME>
con il tuo nome e cognome

Build dell'immagine

2

```
$ docker build -t hello-new-image:1.0 .
Sending build context to Docker daemon 2.048kB
Step 1/2 : FROM ubuntu:15.04
--> d1b55fd07600
Step 2/2 : CMD echo "Hello new image!"
--> Running in d47bf5daca6c
Removing intermediate container d47bf5daca6c
--> 080d4c508a92
Successfully built 080d4c508a92
Successfully tagged hello-new-image:1.0
```

Run del container

4

```
$ docker run hello-new-image:1.0
Hello by Denis Maggiorotto!
```

List delle immagini

3

```
$ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
ubuntu	15.04	d1b55fd07600	4 years ago	131MB
hello-new-image	1.0	080d4c508a92	About a minute ago	131MB

Docker Hub

Creazione di un'utenza su DockerHub (<https://hub.docker.com>)

The image shows the Docker Hub homepage. The top navigation bar includes links for 'Explore', 'Sign In', and 'Pricing', with the 'Sign In' link circled in red. Below the navigation bar, the main heading reads 'Build and Ship any Application Anywhere'. A 'Sign up for Docker Hub' button is visible. An inset on the right shows the 'Welcome Back' login form, which includes fields for 'Docker ID' and 'Password', a 'Sign In' button, and a 'Sign Up' link circled in red.

Sign In

Welcome Back
Sign in with your Docker ID

Docker ID

Password

Sign In

[Forgot Docker ID or Password?](#) | [Sign Up](#)

Condivisione di un'immagine

Tag dell'immagine

1

```
$ docker tag hello-new-image:1.0 <docker hub username>/hello-new-image:1.0
```

Login a Docker Hub

2

```
$ docker login  
Login Succeeded
```

Ricordati di sostituire
! **<docker hub username>** con
il tuo username creato su
Docker Hub

Push dell'immagine

3

```
$ docker push <docker hub username>/hello-new-image:1.0  
The push refers to repository [docker.io/dennydg11/hello-new-image]  
5f70bf18a086: Mounted from lorel/docker-stress-ng  
ed58a6b8d8d6: Pushed  
84cc3d400b0d: Pushed  
3cbe18655eb6: Pushed  
1.0: digest: sha256:11d580acb582a49a9b21ff746b28cfd2ebc33fe83c4113667f8017a0d373d341 size: 1149
```

Verifica l'immagine su Docker Hub



Welcome Back
Sign in with your Docker ID

[Sign In](#)

[Explore](#)[Repositories](#)[Organizations](#)[Get Help](#)dennydgl1

dennydgl1

[Create Repository](#)

dennydgl1 / **hello-new-image**

Updated 15 minutes ago

☆ 0

↓ 1

🌐 PUBLIC

Organizations

 sunnyvale

[View All Orgs](#)

Creazione di un container con l'immagine del compagno

Run di un container scaricando l'immagine di un collega

1

```
$ docker run <docker hub username>/hello-new-image:1.0
```

Ricordati di sostituire

! **<docker hub username>** con
• lo username creato su Docker
Hub dal collega

LAB

https://github.com/sunnyvale-academy/ITS-ICT_Containers

Lab 04 - Docker images and containers